

Why Do We Test Software?

A Motivating Example

192-211 Automated Software Testing • Week 1

International College, Siam University

The Request

Given a list of numbers, I want to know the **smallest** and the **largest** numbers there!

– a customer

The Developer

Yes, I can do it!!

Step 1: A Skeleton

```
class NumFinder:  
    def find(self, nums):  
        pass
```

Step 1: A Skeleton

```
class NumFinder:  
    def find(self, nums):  
        pass
```

A class with one method that will scan the list.

Step 2: First Attempt

```
class NumFinder:
    def __init__(self):
        self.smallest = 0
        self.largest = 0

    def find(self, nums):
        for n in nums:
            if n < self.smallest:
                self.smallest = n
            elif n > self.largest:
                self.largest = n
```

Scan every number; track the smallest and largest seen.

Step 3: Sensible Starting Values

```
class NumFinder:
    def __init__(self):
        self.smallest = float('inf')    # start high
        self.largest = float('-inf')    # start low

    def find(self, nums):
        for n in nums:
            if n < self.smallest:
                self.smallest = n
            elif n > self.largest:
                self.largest = n
```

Python has no `Integer.MAX_VALUE`; we use `float('inf')` and `float('-inf')` so the first number always replaces the starting value.

Almost Done

I'm done! Let me just do a small check...

A Quick Check

```
nf = NumFinder()
nf.find([4, 25, 7, 9])

print(nf.largest)    # expect 25
print(nf.smallest)   # expect 4
```

It Runs...

25

4

Yes, it works!

BOOM

...but does it *really* work?

A Different Input

If I pass **[4, 3, 2, 1]**, the program goes crazy!!

The Crazy Output

```
nf = NumFinder()
nf.find([4, 3, 2, 1])

print(nf.largest)    # -inf ???
print(nf.smallest)  # 1
```

```
-inf
1
```

My bad... :/

Why? The elif Trap

For a descending list, every number is a new smallest, so the `elif` branch never runs and `largest` is never updated.

```
# nums = [4, 3, 2, 1],  smallest=inf, largest=-inf
# n=4 : 4 < inf    -> smallest=4    (elif skipped)
# n=3 : 3 < 4      -> smallest=3    (elif skipped)
# n=2 : 2 < 3      -> smallest=2    (elif skipped)
# n=1 : 1 < 2      -> smallest=1    (elif skipped)
# largest stays -inf  <-- the bug
```

The Fix: Two Independent Checks

```
def find(self, nums):  
    for n in nums:  
        if n < self.smallest:  
            self.smallest = n  
        if n > self.largest:    # separate if, not elif  
            self.largest = n
```

```
find([4, 3, 2, 1]) -> largest = 4, smallest = 1 (correct)
```

A one-word change (`elif` → `if`) fixes it – but how would we have *noticed* without checking?

The Lesson

Mistakes can happen...

We should **rigorously** test our software!

What This Course Is About

A single overlooked case broke a ten-line program. Real systems have thousands of cases.

In this course you will learn to test software **systematically and automatically**, so bugs like this are caught *before* anyone else sees them:

- ▶ Write tests that check many inputs at once
- ▶ Cover edge cases and boundaries on purpose
- ▶ Run every test automatically on every change

Let's begin.